

EventBusClient

v. 0.2.0

Nguyen Huynh Tri Cuong

20.02.2026

Contents

1	Introduction	1
2	Description	2
2.1	Overview	2
2.2	Features	2
2.3	Architecture	2
2.4	Installation	3
2.5	Dependencies	3
2.6	Directory Structure	3
2.7	Configuration	4
2.8	Usage	5
2.8.1	Initialization	5
2.8.2	Publishing Messages	6
2.8.3	Subscribing to Messages	6
2.8.4	Setting Unroutable Message Policy	6
2.9	Example: Producer and Consumer	7
2.10	Example: Custom Plugin Structure	7
2.11	Example: Built-in Plugins	7
2.12	Example: Built-in Configuration	8
2.13	Example: Custom Exchange Handler	8
2.14	Example: Custom Message Class	8
2.15	Example: Custom Serializer	9
3	__init__.py	10
4	connection.py	11
4.1	Method: is_connected	11
4.2	Method: reset_loop	11
4.3	Method: register_exchange_handler	11
4.4	Method: unregister_exchange_handler	11
5	constants.py	12
5.1	Class: SocketType	12
5.2	Class: String	12
6	event_bus_client.py	13
6.1	Function: get_running_loop_or_none	13
6.2	Function: is_on_loop	13
6.3	Class: EventBusClient	13

6.3.1	Method: getVersion	13
6.3.2	Method: getVersionDate	13
6.3.3	Method: wait_on_general_topic_for_one	13
6.3.4	Method: wait_on_general_topic_for_many	14
6.3.5	Method: wait_on_cache_for_one	14
6.3.6	Method: wait_on_cache_for_many	15
6.3.7	Method: constructor_params_from_config	16
6.3.8	Method: is_connected	16
6.3.9	Method: build_routing	16
6.3.10	Method: build_routing_key	17
6.3.11	Method: start_background_loop	17
6.3.12	Method: stop_background_loop	17
6.3.13	Method: from_config_sync	17
6.3.14	Method: connect_sync	18
6.3.15	Method: send_sync_compat	18
6.3.16	Method: send_sync	18
6.3.17	Method: on_sync	19
6.3.18	Method: off_sync	19
6.3.19	Method: wait_until_ready_sync	19
6.3.20	Method: announce_ready_sync	20
6.3.21	Method: close_sync	20
6.3.22	Method: pop_unroutables	20
7	alternate_exchange_sample.py	21
7.1	Function: main	21
7.2	Class: TestMessage	21
7.2.1	Method: get_value	21
8	basic_async_producer_consumer_sample.py	22
8.1	Function: run_process	22
8.2	Class: TestMessage	22
8.2.1	Method: from_data	22
8.2.2	Method: get_value	22
9	basic_sync_producer_consumer_sample.py	23
9.1	Function: producer_process	23
9.2	Function: consumer_process	23
9.3	Function: main	23
9.4	Class: TestMessage	23
9.4.1	Method: from_data	23
9.4.2	Method: get_value	23
10	custom_message_sample.py	24
10.1	Function: main	24
10.2	Class: UserMessage	24
10.2.1	Method: get_value	24
10.2.2	Method: to_dict	24

10.3	Class: OrderMessage	24
10.3.1	Method: get_value	24
10.3.2	Method: total_items	24
10.3.3	Method: get_value	24
10.4	Class: ShipmentMessage	24
10.4.1	Method: get_value	25
10.5	Class: EventMessage	25
10.5.1	Method: get_value	25
11	error_handling_sample.py	26
11.1	Class: TaskMessage	26
11.1.1	Method: get_value	26
11.2	Class: RetryableClient	26
12	multiple_subscriptions_sample.py	27
12.1	Function: main	27
12.2	Class: SensorMessage	27
12.2.1	Method: get_value	27
12.3	Class: AlertMessage	27
12.3.1	Method: get_value	27
12.4	Class: SystemMessage	27
12.4.1	Method: get_value	27
13	rendezvous_sample.py	28
13.1	Function: main	28
13.2	Class: TaskMessage	28
13.2.1	Method: get_value	28
13.3	Class: ResultMessage	28
13.3.1	Method: get_value	28
14	request_reply_sample.py	29
14.1	Function: main	29
14.2	Class: RequestMessage	29
14.2.1	Method: get_value	29
14.3	Class: ResponseMessage	29
14.3.1	Method: get_value	29
14.4	Class: RpcServer	29
15	wait_mode_sample.py	30
15.1	Function: main	30
15.2	Class: OrderMessage	30
15.2.1	Method: get_value	30
16	base.py	31
16.1	Method: configure_unroutable	31
16.2	Method: reset_loop	31
16.3	Method: with_alternate_exchange	31
17	fanout_handler.py	33

18	topic_handler.py	34
19	x_rtopic_handler.py	35
20	base_message.py	36
20.1	Method: from_data	36
20.2	Method: get_value	36
21	basic_message.py	37
21.1	Method: to_dict	37
21.2	Method: from_dict	37
21.3	Method: from_data	37
21.4	Method: get_value	38
22	control_message.py	39
22.1	Method: get_value	39
22.2	Method: from_data	39
23	__init__.py	40
24	dict_msg.py	41
25	float32_msg.py	42
26	float64_msg.py	43
27	header.py	44
28	int32_msg.py	45
29	msg.py	46
30	string_msg.py	47
31	uint32_msg.py	48
32	__init__.py	49
33	listener_event_indexer.py	50
33.1	Class: ListenerEventIndexer	50
34	listener_event_msg.py	52
35	dict_message.py	53
35.1	Method: get_value	53
36	plugin_loader.py	54
36.1	Class: ConfigValidator	54
36.1.1	Method: validate	54
36.2	Class: PluginLoader	54
36.2.1	Method: get_serializer	54
36.2.2	Method: get_exchange_handler	54
36.2.3	Method: get_message	55

36.2.4	Method: load_config	55
37	publisher.py	56
38	qlogger.py	57
38.1	Class: ColorFormatter	57
38.1.1	Method: format	57
38.2	Class: QFileHandler	57
38.2.1	Method: get_log_path	57
38.2.2	Method: get_config_supported	58
38.3	Class: QDefaultFileHandler	58
38.3.1	Method: get_log_path	58
38.3.2	Method: get_config_supported	58
38.4	Class: QConsoleHandler	58
38.4.1	Method: get_config_supported	59
38.5	Class: QLogger	59
38.5.1	Method: get_logger	59
38.5.2	Method: set_handler	59
38.5.3	Method: get_handler	59
39	rendezvous.py	60
40	base_serializer.py	61
40.1	Method: serialize	61
40.2	Method: deserialize	61
41	json_serializer.py	62
41.1	Method: serialize	62
41.2	Method: deserialize	62
42	pickle_serializer.py	64
42.1	Method: serialize	64
42.2	Method: deserialize	64
43	protobuf_serializer.py	65
43.1	Method: serialize	65
43.2	Method: deserialize	65
44	startup_policy.py	66
44.1	Function: resolve_message_cls	66
44.2	Function: build_policy_from_item	66
44.3	Function: build_policy_from_config	67
44.4	Class: StartupPolicy	67
44.5	Class: NoWait	67
44.6	Class: FixedDelay	67
44.7	Class: HandshakeBarrier	67
44.8	Class: PanelControlLegacyByAlias	68
44.9	Class: GeneralCacheStartupPolicy	68
44.10	Class: ConfigureUnroutablePolicy	68

44.11	Class: PolicyChain	68
45	subscriber.py	69
45.1	Method: cache	69
45.2	Method: routing_key	69
45.3	Method: callback	69
46	subscription_cache.py	70
46.1	Method: append	70
46.2	Method: get	70
46.3	Method: pop	70
46.4	Method: pop_nothrow	71
46.5	Method: peek_nothrow	71
46.6	Method: peek	71
46.7	Method: wait_for	71
46.8	Method: drain	72
46.9	Method: peek_last	72
46.10	Method: wait_for_one	72
46.11	Method: wait_for_many	72
47	utils.py	74
47.1	Class: Singleton	74
47.2	Class: DictToClass	74
47.2.1	Method: validate	74
47.3	Class: Utils	74
47.3.1	Method: get_all_descendant_classes	74
47.3.2	Method: get_all_sub_classes	75
47.3.3	Method: is_valid_host	75
47.3.4	Method: caller_name	75
47.3.5	Method: load_library	75
47.3.6	Method: is_ascii_or_unicode	75
47.4	Class: Job	76
47.4.1	Method: stop	76
47.4.2	Method: run	76
47.5	Class: ResultType	76
47.6	Class: ResponseMessage	76
47.6.1	Method: get_json	76
47.6.2	Method: get_data	76
47.6.3	Method: create_from_string	76
48	wait_mode.py	77
49	Glossary	78
50	Appendix	79
51	History	80

Chapter 1

Introduction

The component **EventBusClient** provides a modular, high-level messaging framework built on top of the message broker **RabbitMQ**. It abstracts low-level standard protocol operations and adds:

- Plugin support (custom serializers, handlers, message types)
- Dynamic configuration
- Reconnect and lifecycle management

It enables multiple projects to reuse a consistent event bus API without rewriting **RabbitMQ** logic.

The **EventBusClient** is part of a test automation framework called **RobotFramework AIO**, but can also be installed and used stand-alone.

RabbitMQ is an open-source message broker that enables different parts of an application or separate applications to communicate with each other by sending and receiving messages asynchronously. It acts as an intermediary between producers (which send messages) and consumers (which receive and process messages). **RabbitMQ** decouples the sender and receiver, allowing them to operate independently and at their own pace.

Messages are stored in queues until they can be processed, ensuring reliability even if a consumer is temporarily unavailable.

RabbitMQ terms

Term	Description
Producer	Application or service that sends messages
Consumer	Application or service that receives and processes messages
Queue	Buffer that temporarily stores messages
Exchange	Routes messages from producers to queues based on routing rules
Binding	Link between an exchange and a queue, defining how messages are routed
Routing Key	Address-like string used to decide how messages are routed
AMQP	Advanced Message Queuing Protocol, the standard protocol RabbitMQ uses

For more detailed information about these components and frameworks please refer to the corresponding [homepages](#).

Chapter 2

Description

2.1 Overview

The **EventBusClient** provides a high-level API for interacting with a **RabbitMQ**-based event bus system. It abstracts low-level AMQP details and supports dynamic loading of project-specific plugins for serializers, exchange handlers, and message structures. This client is designed for modularity and ease of use across multiple projects.

2.2 Features

- **Dynamic plugin loading:** Load custom serializers, exchange handlers, and message classes at runtime from a configurable plugins directory.
- **Thread-safe publishing:** Supports safe publishing from multiple threads with minimal locking overhead.
- **Automatic reconnect:** Automatically reconnects to **RabbitMQ** after connection loss (if enabled in configuration).
- **Pluggable architecture:** Supports both built-in and project-specific extensions.
- **Supports JSON, Protobuf, Pickle serializers.**

2.3 Architecture

The **EventBusClient** is built around a modular architecture that allows for easy extension and customization. It consists of the following components:

- **Base Classes:** Define interfaces for exchange handlers, serializers, and messages.
- **Plugins Directory:** Contains custom implementations of the base classes.
- **Configuration File:** Specifies the plugins path, **RabbitMQ** connection details, and selected implementations.
- **EventBusClient Class:** The main class that interacts with **RabbitMQ** and manages plugins.
- **Message Classes:** Define the structure of messages sent and received over the event bus.
- **Exchange Handlers:** Handle the declaration and publishing of messages to **RabbitMQ** exchanges.
- **Serializers:** Convert messages to and from byte streams for transmission over **RabbitMQ**.

The client uses a combination of built-in plugins and user-defined plugins to provide flexibility in message handling and serialization.

2.4 Installation

To install the **EventBusClient**, use pip to install the package from PyPI:

```
pip install eventbusclient
```

Ensure you have **RabbitMQ** server running and accessible at the specified host and port in the configuration file.

2.5 Dependencies

The **EventBusClient** requires the following dependencies:

- **pika**: For synchronous **RabbitMQ** communication
- **aio-pika**: Specifically for asynchronous **RabbitMQ** communication
- **protobuf**: For Protocol Buffers serialization (if using **ProtobufSerializer**)
- **jsonpickle**: For JSON serialization (if using **JSONSerializer**)

Ensure these dependencies are installed in your Python environment. You can install them using pip:

```
pip install pika protobuf jsonpickle
```

2.6 Directory Structure

```
project/  
|-- EventBusClient/  
|   |-- event_bus_client.py  
|   |-- connection.py  
|   |-- plugin_loader.py  
|   |-- publisher.py  
|   |-- subscriber.py  
|   |-- qlogger.py  
|   |-- message/  
|   |   |-- base_message.py  
|   |-- exchange_handler/  
|   |   |-- base.py  
|   |   |-- xr_topic_handler.py  
|   |-- serializer/  
|   |   |-- base_serializer.py  
|   |   |-- json_serializer.py  
|   |   |-- protobuf_serializer.py  
|   |   |-- pickle_serializer.py  
|   |-- plugins/  
|   |   |-- mcpi/  
|   |   |   |-- serialize/  
|   |   |   |-- message/  
|   |   |   |-- exchange/  
|   |-- config/  
|   |   |-- config.jsonp  
|-- app.py
```

2.7 Configuration

The `config.jsonp` file controls all aspects of the **EventBusClient**, including plugin paths, **RabbitMQ** connection, and selected implementations.

Configuration Template (EventBusClient/config/config.jsonp.template)

Note: This is JSON with comments (.jsonp). Remove comments (// , / */) if your parser does not support them.*

```
{
  // plugins_path: Path to the directory containing plugin modules.
  "plugins_path": "./plugins",

  // host: Hostname or IP address for the server connection.
  "host": "localhost",

  // port: Port number for the server connection.
  "port": 5672,

  // serializer: Serialization method for message data (e.g., PickleSerializer).
  "serializer": "PickleSerializer",

  // exchange_handler: Handler type for message exchange (e.g., TopicExchangeHandler).
  "exchange_handler": "TopicExchangeHandler",

  // auto_reconnect: Automatically reconnect if the connection is lost (true/false).
  "auto_reconnect": true,

  // qos_prefetch: Number of messages to prefetch for Quality of Service.
  "qos_prefetch": 10,

  // general_cache_policy: "off" | "on_connect" | "on_demand".
  "general_cache_policy": "off",

  // general_routing_keys: Routing keys for general messages.
  "general_routing_keys": "general",

  // general_message_cls: Message class for general messages.
  "general_message_cls": "BaseMessage",

  // logger_name: Name of the logger instance.
  "logger_name": "test",

  // logfile: Path to the log file.
  "logfile": "test.log",

  // loglevel: Logging level for the logger instance.
  "loglevel": "INFO"
}
```

How to Use

1. Copy the template above into your project as `config.jsonp`.
2. Adjust each value to match your environment and requirements.
3. Save the file and restart your application to apply changes.

Field Descriptions

Field	Required	Type	Default	Description / Options
<code>plugins_path</code>	Optional	String	<code>./plugins</code>	Path to the plugins directory (relative or absolute).
<code>host</code>	Optional	String	<code>"localhost"</code>	Server hostname or IP address for the broker connection.
<code>port</code>	Optional	Integer	<code>5672</code>	Server port for the broker connection.
<code>serializer</code>	Mandatory	String	<code>(none)</code>	Serialization strategy for message bodies (e.g., <code>PickleSerializer</code>). Must be available to both producer and consumer sides.
<code>exchange_handler</code>	Mandatory	String	<code>(none)</code>	Exchange handler implementation (e.g., <code>TopicExchangeHandler</code>). Determines exchange type and binding logic.
<code>auto_reconnect</code>	Optional	Boolean	<code>true</code>	Reconnect automatically if the connection is lost.
<code>qos_prefetch</code>	Optional	Integer	<code>10</code>	Prefetch count for consumer QoS (how many unacked messages a consumer can receive).
<code>general_cache_policy</code>	Optional	String	<code>"off"</code>	Policy controlling the general message cache: <code>"off"</code> – disabled; <code>"on_connect"</code> – start caching upon connection establishment; <code>"on_demand"</code> – start caching only when first requested.
<code>general_routing_keys</code>	Optional	String (or list)	<code>"general"</code>	Routing key or keys used for the general cache subscription (supports topic patterns).
<code>general_message_cls</code>	Optional	String	<code>"BaseMessage"</code>	Message class to use for general-cache decoding. Provide a resolvable name or dotted path if needed.
<code>logger_name</code>	Optional	String	<code>"event_bus"</code>	Name of the logger instance to use/create.
<code>logfile</code>	Optional	String / Null	<code>(none)</code>	Path to a log file. If omitted or null, logs go to the default handlers (e.g., console).
<code>loglevel</code>	Optional	String	<code>"INFO"</code>	Logging threshold (<code>"DEBUG"</code> , <code>"INFO"</code> , <code>"WARNING"</code> , <code>"ERROR"</code> , <code>"CRITICAL"</code>).

2.8 Usage

2.8.1 Initialization

Create an `EventBusClient` instance from configuration:

```
from EventBusClient.event_bus_client import EventBusClient

client = await EventBusClient.from_config("./config/config.jsonp")
```

2.8.2 Publishing Messages

Send a message using the configured serializer and exchange handler asynchronously:

```
msg = ListenerEventMsg(event="start_cycle", timestamp=1234567890)
await client.send("zone1.topic.start", msg, header)
```

Enable thread-safe publish from a non-async thread:

```
client.send("zone1.topic.alert", msg, header, threadsafe=True)
```

Send a message synchronously using the corresponding method:

```
client.send_sync("zone1.topic.sync", msg, header)
```

2.8.3 Subscribing to Messages

Subscribe to a routing key and handle incoming messages:

```
async def on_message(msg: ListenerEventMsg, header):
    print(f"Received message: {msg}, header: {header}")

await client.on("zone1.#", ListenerEventMsg, on_message)
```

Subscribe to a routing key and handle incoming messages synchronously using `on_sync`:

```
def on_message_sync(msg: ListenerEventMsg, header):
    print(f"Received message: {msg}, header: {header}")

client.on_sync("zone1.#", ListenerEventMsg, on_message_sync)
```

2.8.4 Setting Unroutable Message Policy

You can configure how the client handles unroutable messages by setting a startup policy. For example:

```
from EventBusClient.event_bus_client import EventBusClient
from EventBusClient.startup_policy import ConfigureUnroutablePolicy

config_unrout = ConfigureUnroutablePolicy(
    mode="return",                # "drop" (default), "alternate-exchange", or "return"
    alternate_exchange=None,      # Name of alternate exchange if mode is ↩
    ↪ "alternate-exchange"
    on_unroutable="log",          # "log" (default), "raise", "cache", or "callback"
    on_unroutable_callback=None   # User-defined callback function if on_unroutable is ↩
    ↪ "callback"
)

client = await EventBusClient.from_config(config_path, startup_policy=config_unrout)
```

- `mode`: (*optional, str, default: "drop"*)
Determines how unroutable messages are handled: "drop" (discard), "alternate-exchange" (send to alternate exchange), or "return" (return to sender).
- `alternate_exchange`: (*optional, str or None, default: None*)
The name of the alternate exchange to use if mode is "alternate-exchange".
- `on_unroutable`: (*optional, str, default: "log"*)
The action to take on unroutable messages: "log" (log the event), "raise" (raise an exception), "cache" (store for later), or "callback" (invoke a user callback).
- `on_unroutable_callback`: (*optional, callable or None, default: None*)
A user-defined function to call when an unroutable message is received (only if `on_unroutable` is set to "callback").

2.9 Example: Producer and Consumer

This example (from `EventBusClient/examples/basic_async_producer_consumer_sample.py`) starts a producer and a consumer in separate processes:

```
# Start consumer
consumer = Process(target=run_process, args=(consumer_process, config_path))
consumer.start()

# Start producer
producer = Process(target=run_process, args=(producer_process, config_path))
producer.start()
# Wait for both to finish
producer.join()
consumer.join()
```

You can find the complete example in the `EventBusClient/examples` directory. Run `run_sample.bat` (Windows) to execute the example.

2.10 Example: Custom Plugin Structure

To create a custom plugin, follow these steps:

- Create a directory for your plugin in the configured plugins path.
- Implement the required interfaces (e.g., `BaseExchangeHandler`, `BaseMessage`, `BaseSerializer`).
- Register your plugin in the `config.jsonp` file.
- Ensure your plugin is discoverable by the **EventBusClient**.
- Place your plugin code in the plugins directory, following the structure:
- For example, if your plugin is named `CustomPlugin`, the directory structure should look like this:

```
plugins/
|-- CustomPlugin/
    |-- __init__.py
    |-- custom_exchange_handler.py
    |-- custom_message.py
    |-- custom_serializer.py
```

2.11 Example: Built-in Plugins

The **EventBusClient** comes with several built-in plugins that can be used directly or extended:

- **XRTopicExchangeHandler**: Handles topic exchanges with **RabbitMQ**.
- **ListenerEventMsg**: A default message class for listener events.
- **PickleSerializer**: Serializes messages using Python's `Pickle` module.
- **JSONSerializer**: Serializes messages to JSON format.
- **ProtobufSerializer**: Serializes messages using Protocol Buffers.

These plugins can be used as-is or extended to create custom functionality.

2.12 Example: Built-in Configuration

The built-in configuration file `config.jsonp` provides a starting point for configuring the **EventBusClient**:

```
{
  "plugins_path": "./plugins",
  "host": "localhost",
  "port": 5672,
  "serializer": "PickleSerializer",
  "exchange_handler": "XRTopicExchangeHandler",
  "message_class": "ListenerEventMsg",
  "threadsafe_publish": true,
  "auto_reconnect": true,
  "qos_prefetch": 10
}
```

This configuration specifies the plugins path, **RabbitMQ** connection details, serializer, exchange handler, message class, and other options.

2.13 Example: Custom Exchange Handler

To create a custom exchange handler, implement the required interface and place it in the plugins directory. For example, a custom exchange handler might look like this:

```
from event_bus.plugins import BaseExchangeHandler
class CustomExchangeHandler(BaseExchangeHandler):
    def declare_exchange(self, channel):
        # Custom exchange declaration logic
        pass

    def publish(self, channel, routing_key, message):
        # Custom publish logic
        pass

# Register the plugin in config.jsonp
{
  "plugins_path": "./plugins",
  "exchange_handler": "CustomExchangeHandler"
}
```

2.14 Example: Custom Message Class

To create a custom message class, define it in your project and register it in the configuration:

```
from event_bus.plugins import BaseMessage
class CustomMessage(BaseMessage):
    def __init__(self, data):
        self.data = data

    @classmethod
    def from_data(cls, data):
        pass

    @abstractmethod
    def get_value(self):
        pass

# Register the message class in config.jsonp
{
  "plugins_path": "./plugins",
  "message_class": "CustomMessage"
}
```

2.15 Example: Custom Serializer

To create a custom serializer, implement the required interface and place it in the plugins directory. For example, a custom serializer might look like this:

```
from event_bus.plugins import BaseSerializer
class CustomSerializer(BaseSerializer):
    def serialize(self, obj):
        # Custom serialization logic
        pass

    def deserialize(self, data):
        # Custom deserialization logic
        pass
# Register the plugin in config.jsonp
{
    "plugins_path": "./plugins",
    "serializer": "CustomSerializer"
}
```


Chapter 3

`__init__.py`

Chapter 4

connection.py

ConnectionManager: Manages RabbitMQ connections, channels, and exchanges.

4.1 Method: `is_connected`

Check if the connection to RabbitMQ is established.

Returns:

/ Type: bool /

True if connected, False otherwise.

4.2 Method: `reset_loop`

4.3 Method: `register_exchange_handler`

Register an exchange handler to handle messages from the exchange.

Arguments:

- `handler`

/ Condition: required / Type: ExchangeHandler /

The exchange handler to register. It should be an instance of ExchangeHandler or its subclasses.

4.4 Method: `unregister_exchange_handler`

Unregister an exchange handler.

Arguments:

- `handler`

/ Condition: required / Type: ExchangeHandler /

The exchange handler to unregister. It should be an instance of ExchangeHandler or its subclasses.

Chapter 5

constants.py

5.1 Class: SocketType

5.2 Class: String

Chapter 6

event_bus_client.py

6.1 Function: get_running_loop_or_none

6.2 Function: is_on_loop

6.3 Class: EventBusClient

EventBusClient: Client for interacting with the event bus system.

6.3.1 Method: getVersion

Returns the version of EventBusClient as string.

6.3.2 Method: getVersionDate

Returns the version date of EventBusClient as string.

6.3.3 Method: wait_on_general_topic_for_one

Wait for a specific message on the general topic.

Arguments:

- `msg`
/ *Condition*: required / *Type*: Any /
The message to wait for. This can be any object that can be compared for equality.
- `timeout`
/ *Condition*: optional / *Type*: float / *Default*: 30.0 /
The maximum time to wait for the message, in seconds. Defaults to 30.0 seconds.
- `interval`
/ *Condition*: optional / *Type*: float / *Default*: 0.1 /
Note: This parameter is accepted for compatibility with older interfaces but is not used in the current implementation.
- `asynchronous`
/ *Condition*: optional / *Type*: bool / *Default*: False /
If True, the wait will be performed asynchronously using a ThreadPoolExecutor. Defaults to False.

Returns:

/ *Type*: bool /
True if the message was received within the timeout period, False otherwise.

6.3.4 Method: wait_on_general_topic_for_many

Wait for multiple specific messages on the general topic.

Arguments:

- `msgs`
/ Condition: required / Type: List[Any] /
 The list of messages to wait for. Each message can be any object that can be compared for equality.
- `mode`
/ Condition: optional / Type: int / Default: WaitMode.ALL_IN_GIVEN_ORDER.value /
 The mode for waiting:
 - `WaitMode.ALL_IN_GIVEN_ORDER`: Wait for all messages in the order they are provided.
 - `WaitMode.ALL_IN_RANDOM_ORDER`: Wait for all messages in any order.
 - `WaitMode.ANY`: Wait for any one of the messages.

Defaults to `WaitMode.ALL_IN_GIVEN_ORDER.value`.

- `timeout`
/ Condition: optional / Type: float / Default: 30.0 /
 The maximum time to wait for the messages, in seconds. Defaults to 30.0 seconds.
- `interval`
/ Condition: optional / Type: float / Default: 0.1 /
Note: This parameter is accepted for compatibility with older interfaces but is not used in the current implementation.
- `asynchronous`
/ Condition: optional / Type: bool / Default: False /
 If True, the wait will be performed asynchronously using a `ThreadPoolExecutor`. Defaults to False.

Returns:

/ Type: List[int] /
 A list of indices of the messages that were received, based on the specified mode.

6.3.5 Method: wait_on_cache_for_one

Wait for a specific message in the given subscription cache.

Arguments:

- `cache`
/ Condition: required / Type: SubscriptionCache /
 The subscription cache to wait on. This should be an instance of `SubscriptionCache`.
- `msg`
/ Condition: required / Type: Any /
 The message to wait for. This can be any object that can be compared for equality.
- `timeout`
/ Condition: optional / Type: float / Default: 30.0 /
 The maximum time to wait for the message, in seconds. Defaults to 30.0 seconds.
- `interval`
/ Condition: optional / Type: float / Default: 0.1 /
Note: This parameter is accepted for compatibility with older interfaces but is not used in the current implementation.

- asynchronous

/ *Condition*: optional / *Type*: bool / *Default*: False /

If True, the wait will be performed asynchronously using a ThreadPoolExecutor. Defaults to False.

Returns:

/ *Type*: bool /

True if the message was received within the timeout period, False otherwise.

6.3.6 Method: wait_on_cache_for_many

Wait for multiple specific messages in the given subscription cache.

Arguments:

- cache

/ *Condition*: required / *Type*: SubscriptionCache /

The subscription cache to wait on. This should be an instance of SubscriptionCache.

- msgs

/ *Condition*: required / *Type*: List[Any] /

The list of messages to wait for. Each message can be any object that can be compared for equality.

- mode

/ *Condition*: optional / *Type*: int / *Default*: WaitMode.ALL_IN_GIVEN_ORDER.value /

The mode for waiting:

- WaitMode.ALL_IN_GIVEN_ORDER: Wait for all messages in the order they are provided.
- WaitMode.ALL_IN_RANDOM_ORDER: Wait for all messages in any order.
- WaitMode.ANY: Wait for any one of the messages.

Defaults to WaitMode.ALL_IN_GIVEN_ORDER.value.

- timeout

/ *Condition*: optional / *Type*: float / *Default*: 30.0 /

The maximum time to wait for the messages, in seconds. Defaults to 30.0 seconds.

- interval

/ *Condition*: optional / *Type*: float / *Default*: 0.1 /

Note: This parameter is accepted for compatibility with older interfaces but is not used in the current implementation.

- asynchronous

/ *Condition*: optional / *Type*: bool / *Default*: False /

If True, the wait will be performed asynchronously using a ThreadPoolExecutor. Defaults to False.

Returns:

/ *Type*: List[int] /

A list of indices of the messages that were received, based on the specified mode.

6.3.7 Method: constructor_params_from_config

Returns a tuple of parameters for the EventBusClient constructor, loaded from config and provided arguments.

Arguments:

- `config_path`
/ Condition: required / Type: str /
Path to the configuration file in JSONP format. This file should contain the necessary configuration for the event bus client, including exchange handler and serializer settings.
- `startup_policy`
/ Condition: optional / Type: StartupPolicy / Default: None /
The startup policy to use for the client. If not provided, no startup policy will be used.
- `zone_id`
/ Condition: optional / Type: str / Default: None /
The zone ID for the client. If not provided, no zone ID will be used.
- `alias`
/ Condition: optional / Type: str / Default: None /
The alias for the client. If not provided, no alias will be used.
- `startup_policies`
/ Condition: optional / Type: list[StartupPolicy] / Default: None /
A list of startup policies to use for the client. If provided, these will be combined into a PolicyChain.

Returns:

/ Type: dict /
A dictionary of parameters to be used for the EventBusClient constructor.

6.3.8 Method: is_connected

Check if the client is currently connected to the event bus.

Returns:

/ Type: bool /
True if the client is connected, False otherwise.

6.3.9 Method: build_routing

Backward-compatible alias for build_routing_key.

Arguments:

- `path`
/ Condition: required / Type: str /
The components of the routing key. Each component will be joined with a dot (.) to form the final routing key.

Returns:

/ Type: str /
The constructed routing key as a string.

6.3.10 Method: build_routing_key

Build a routing key from the given path components.

Arguments:

- path
/ Condition: required / Type: str /

The components of the routing key. Each component will be joined with a dot (.) to form the final routing key.

Returns:

- str
/ Type: str /

The constructed routing key as a string.

6.3.11 Method: start_background_loop

Start a dedicated asyncio loop in a background thread if not already running. Safe to call multiple times.

This method is useful for blocking synchronous APIs that need to run in a separate thread to avoid blocking the main thread, especially in environments where the main thread is already running an event loop (e.g., GUI applications, web servers).

This method will create a new thread that runs an asyncio event loop, allowing you to submit coroutines for execution without blocking the main thread. It also ensures that the loop is ready before returning. It will not start a new loop if one is already running in the background.

Arguments:

- loop_name
/ Condition: optional / Type: str / Default: "EventBusClientLoop" /

The name of the background thread running the asyncio loop. Defaults to "EventBusClientLoop".

6.3.12 Method: stop_background_loop

Stop the background loop (if we created it) and join the thread.

This method is useful for cleaning up resources when the client is no longer needed.

Arguments:

- timeout
/ Condition: optional / Type: float / Default: 3.0 /

The maximum time to wait for the background loop to stop. Defaults to 3.0 seconds.

6.3.13 Method: from_config_sync

Create an EventBusClient instance from a configuration file synchronously.

Arguments:

- path
/ Condition: required / Type: str /

Path to the configuration file in JSONP format. This file should contain the necessary configuration for the event bus client, including exchange handler and serializer settings.

Returns:

/ Type: EventBusClient /

An instance of EventBusClient initialized with the configuration from the specified path.

6.3.14 Method: connect_sync

Blocking connect that spins a background loop if needed.

Arguments:

- `host`
/ *Condition*: optional / *Type*: str / *Default*: "localhost" /
The hostname of the event bus server. Defaults to "localhost".
- `port`
/ *Condition*: optional / *Type*: int / *Default*: 5672 /
The port number of the event bus server. Defaults to 5672.
- `prefetch_count`
/ *Condition*: optional / *Type*: int / *Default*: 10 /
The number of messages to prefetch from the server. This controls how many messages can be sent to the client before they are acknowledged. Defaults to 10.
- `timeout`
/ *Condition*: optional / *Type*: float / *Default*: 30.0 /
The maximum time to wait for the connection to be established. Defaults to 30.0 seconds.

6.3.15 Method: send_sync_compat

Backward-compatible blocking send wrapper.

6.3.16 Method: send_sync

Blocking send wrapper.

Arguments:

- `routing_key`
/ *Condition*: required / *Type*: str /
The routing key used to route the message to the appropriate subscribers.
- `message`
/ *Condition*: required / *Type*: BaseMessage /
The message to be sent. It should be an instance of BaseMessage or its subclasses.
- `headers`
/ *Condition*: optional / *Type*: dict | None /
Additional headers to include with the message. This can be used for metadata or routing information.
- `threadsafe`
/ *Condition*: optional / *Type*: bool / *Default*: False /
If True, the message will be sent in a threadsafe manner. Defaults to False.

Returns:

/ *Type*: None /

This method does not return any value. It sends the message to the event bus and returns immediately.

6.3.17 Method: on_sync

Blocking subscribe wrapper. Returns SubscriptionCache to use with get()/wait_for()/drain().

Arguments:

- `routing_key`
/ Condition: required / Type: str /
 The routing key to subscribe to. Messages with this routing key will be routed to the callback.
- `message_cls`
/ Condition: required / Type: Type[BaseMessage] /
 The class of the message to subscribe to. The callback will receive messages of this type.
- `callback`
/ Condition: optional / Type: Callable[[BaseMessage], Awaitable[None]] /
 The callback function to be called when a message is received. It should accept a single argument of type BaseMessage or its subclasses and return an awaitable (e.g., a coroutine).
- `cache_size`
/ Condition: optional / Type: int / Default: 200 /
 The size of the cache for storing received messages. This is useful for buffering messages before they are processed by the callback.

Returns: */ Type: SubscriptionCache /*

A SubscriptionCache object that allows you to manage the subscription and access received messages.

6.3.18 Method: off_sync

Blocking unsubscribe wrapper.

Arguments:

- `routing_key`
/ Condition: required / Type: str /
 The routing key to unsubscribe from. Messages with this routing key will no longer be routed to the callback.

Returns:

/ Type: None /

This method does not return any value. It unsubscribes from the specified routing key and returns immediately.

6.3.19 Method: wait_until_ready_sync

Blocking rendezvous wait. Returns True if satisfied before timeout.

Arguments:

- `requirements`
/ Condition: required / Type: dict[str, int] /
 A dictionary where keys are role names and values are the number of instances required for each role.
- `timeout`
/ Condition: optional / Type: float / Default: 5.0 /
 The maximum time to wait for the rendezvous to be satisfied. Defaults to 5.0 seconds.

Returns:

/ Type: bool /

True if the rendezvous requirements are satisfied before the timeout, False otherwise.

6.3.20 Method: `announce_ready_sync`

Blocking announce ready via rendezvous control topic.

Arguments:

- `roles`

/ Condition: required / Type: list[str] /

A list of role names that this instance is ready for. This is used to signal readiness in the rendezvous system.

Returns:

/ Type: None /

This method does not return any value. It announces that the instance is ready for the specified roles.

6.3.21 Method: `close_sync`

Optional: call your async close/teardown then stop the loop. If you don't have an async `close()`, this just stops the loop.

Arguments:

- `timeout`

/ Condition: optional / Type: float / Default: 10.0 /

The maximum time to wait for the close operation to complete. Defaults to 10.0 seconds.

6.3.22 Method: `pop_unroutables`

Get and clear the list of unroutable messages.

Returns:

/ Type: list[dict] /

A list of unroutable message dictionaries that were cached.

Chapter 7

alternate_exchange_sample.py

7.1 Function: main

7.2 Class: TestMessage

Simple test message. eventbusclient-examples-alternate-exchange-sample-testmessage-from-data -----

7.2.1 Method: get_value

Chapter 8

basic_async_producer_consumer_sample.py

8.1 Function: run_process

Helper function to run an async function in a process eventbusclient-examples-basic-async-producer-consumer-sample-

main =====

8.2 Class: TestMessage

8.2.1 Method: from_data

8.2.2 Method: get_value

Chapter 9

basic_sync_producer_consumer_sample.py

9.1 Function: producer_process

9.2 Function: consumer_process

9.3 Function: main

9.4 Class: TestMessage

9.4.1 Method: from_data

9.4.2 Method: get_value

Chapter 10

custom_message_sample.py

10.1 Function: main

10.2 Class: UserMessage

User message with basic validation. eventbusclient-examples-custom-message-sample-usermessage-from-data -----

10.2.1 Method: get_value

10.2.2 Method: to_dict

Convert to dictionary for logging/debugging. eventbusclient-examples-custom-message-sample-orderstatus
=====

10.3 Class: OrderMessage

Order message with enum status and computed properties. eventbusclient-examples-custom-message-sample-ordermessage-from-data -----

10.3.1 Method: get_value

10.3.2 Method: total_items

Total number of items in the order. eventbusclient-examples-custom-message-sample-ordermessage-total-price -----

Total price of the order. eventbusclient-examples-custom-message-sample-ordermessage-is-complete -----

Check if order is in a final state. eventbusclient-examples-custom-message-sample-ordermessage-update-status -----

Return a new message with updated status. eventbusclient-examples-custom-message-sample-addressmessage
=====

Address component of a message. eventbusclient-examples-custom-message-sample-addressmessage-from-data -----

10.3.3 Method: get_value

10.4 Class: ShipmentMessage

Shipment message with nested address objects. eventbusclient-examples-custom-message-sample-shipmentmessage-from-data -----

10.4.1 Method: `get_value`

10.5 Class: `EventMessage`

Event message with proper equality for `wait_for` operations. `eventbusclient-examples-custom-message-sample-eventmessage-from-data` -----

10.5.1 Method: `get_value`

Chapter 11

error_handling_sample.py

11.1 Class: TaskMessage

Simple task message. eventbusclient-examples-error-handling-sample-taskmessage-from-data -----

11.1.1 Method: get_value

11.2 Class: RetryableClient

Wrapper that adds retry logic to EventBusClient operations.

Chapter 12

multiple_subscriptions_sample.py

12.1 Function: main

12.2 Class: SensorMessage

Message from a sensor. eventbusclient-examples-multiple-subscriptions-sample-sensormessage-from-data -----

12.2.1 Method: get_value

12.3 Class: AlertMessage

Alert message for threshold violations. eventbusclient-examples-multiple-subscriptions-sample-alertmessage-from-data -----

12.3.1 Method: get_value

12.4 Class: SystemMessage

System status message. eventbusclient-examples-multiple-subscriptions-sample-systemmessage-from-data -----

12.4.1 Method: get_value

Chapter 13

rendezvous_sample.py

13.1 Function: main

13.2 Class: TaskMessage

Message representing a task to be processed. eventbusclient-examples-rendezvous-sample-taskmessage-from-data ----

13.2.1 Method: get_value

13.3 Class: ResultMessage

Message representing a task result. eventbusclient-examples-rendezvous-sample-resultmessage-from-data -----

13.3.1 Method: get_value

Chapter 14

request_reply_sample.py

14.1 Function: main

14.2 Class: RequestMessage

A request message with correlation ID for tracking. eventbusclient-examples-request-reply-sample-requestmessage-from-data -----

14.2.1 Method: get_value

14.3 Class: ResponseMessage

A response message with correlation ID for matching to requests. eventbusclient-examples-request-reply-sample-responsemessage-from-data -----

14.3.1 Method: get_value

14.4 Class: RpcServer

Simple RPC server that handles requests. eventbusclient-examples-request-reply-sample-rpcclient =====
RPC client that sends requests and waits for responses.

Chapter 15

wait_mode_sample.py

15.1 Function: main

15.2 Class: OrderMessage

Message representing an order with a sequence number. eventbusclient-examples-wait-mode-sample-ordermessage-from-data -----

15.2.1 Method: get_value

Chapter 16

base.py

16.1 Method: `configure_unroutable`

Configure the unroutable message handling policy.

Arguments:

- `policy`
/ *Condition*: required / *Type*: str /
The unroutable message handling policy. Options are "drop", "log", "return", "alternate-exchange".
- `alternate_exchange`
/ *Condition*: optional / *Type*: str /
The name of the alternate exchange to use if the policy is "alternate-exchange".
- `on_unroutable`
/ *Condition*: required / *Type*: str /
The action to take for unroutable messages. Options are "log", "cache", "callback", "raise".
- `unroutable_sink`
/ *Condition*: optional / *Type*: list /
The sink (list) to store unroutable messages if the action is "cache".
- `on_unroutable_callback`
/ *Condition*: optional / *Type*: Callable /
The callback function to invoke for unroutable messages if the action is "callback".

16.2 Method: `reset_loop`

Reset the event loop used by the exchange handler.

Arguments:

- `loop`
/ *Condition*: optional / *Type*: `asyncio.AbstractEventLoop` /
The new event loop to use. If not provided, the current event loop will be used.

16.3 Method: `with_alternate_exchange`

Decorator to set up and finalize alternate exchange configuration around the decorated method.

Arguments:

- `func`
/ *Condition*: required / *Type*: Callable /
The method to be decorated.

Chapter 17

fanout_handler.py

Chapter 18

topic_handler.py

Chapter 19

x_rtopic_handler.py

XRTopicExchangeHandler: Handles x-rtopic exchanges for routing messages based on topics.

Chapter 20

base_message.py

BaseMessage: Abstract base class for messages in the event bus system.

20.1 Method: `from_data`

Create a message instance from raw data.

Arguments:

- `data`
/ *Condition*: required / *Type*: Any /
Raw data to create the message instance.

20.2 Method: `get_value`

Get the value of the message.

Chapter 21

basic_message.py

BasicMessage: A simple message class that extends BaseMessage.

This class can be used to create messages that do not require any additional fields or methods. It inherits all the functionality from BaseMessage and can be used as a placeholder for messages that do not need any specific attributes.

21.1 Method: to_dict

Convert the BasicMessage to a dictionary representation.

Returns:

A dictionary containing the content and headers of the message.

21.2 Method: from_dict

Create a BasicMessage from a dictionary representation.

Arguments:

- data
/ *Condition*: required / *Type*: dict /
A dictionary containing the content and headers of the message.

Returns:

An instance of BasicMessage created from the provided dictionary.

21.3 Method: from_data

Create a BasicMessage from raw data.

Arguments:

- data
/ *Condition*: required / *Type*: str /
The raw data to create the message from. This should be a string representation of the message content.

Returns:

An instance of BasicMessage created from the provided data.

21.4 Method: get_value

Convert the BasicMessage to raw data.

Returns:

A string representation of the message content.

Chapter 22

control_message.py

22.1 Method: get_value

22.2 Method: from_data

Chapter 23

`__init__.py`

The package for standard messages.

Inspired by https://github.com/ros/std_msgs, this package defines messages of primitive data types and multiarrays, that can be reused for many common topics. If a more complex structural message is required, please add it to a different package, e.g. `taf_msgs`.

Chapter 24

dict_msg.py

The dict message module. eventbusclient-message-deprecated-std-msgs-dict-msg-dictmsg =====

The dict message class.

Chapter 25

float32_msg.py

The float32 message module. eventbusclient-message-deprecated-std-msgs-float32-msg-float32msg =====

The float32 message class.

Chapter 26

float64_msg.py

The float64 message module. eventbusclient-message-deprecated-std-msgs-float64-msg-float64msg =====

The float64 message class.

Chapter 27

header.py

The header module. eventbusclient-message-deprecated-std-msgs-header-header =====

The header class.

Chapter 28

int32_msg.py

The int32 message module. eventbusclient-message-deprecated-std-msgs-int32-msg-int32msg =====

The int32 message class.

Chapter 29

msg.py

The base message module. eventbusclient-message-deprecated-std-msgs-msg-msg =====

The base message class. eventbusclient-message-deprecated-std-msgs-msg-msg-serialize -----

Serialize. eventbusclient-message-deprecated-std-msgs-msg-msg-deserialize -----

Deserialize.

Chapter 30

string_msg.py

The string message module. eventbusclient-message-deprecated-std-msgs-string-msg-stringmsg =====

The string message class.

Chapter 31

`uint32_msg.py`

The `uint32` message module. `eventbusclient-message-deprecated-std-msgs-uint32-msg-uint32msg` =====

The `uint32` message class.

Chapter 32

`__init__.py`

The package for TAF messages.

This package defines messages for TAF operations, e.g. rack synchronization, recovery, etc. If just a primitive data type is enough for your use case, please refer to the `std_msgs` package.

Chapter 33

listener_event_indexer.py

The listener event indexer module.

This module implements a compact indexer specialized for the synchronization across multiple TAF instances. The idea is to pack all possible states into one 64-bit integer (hence 'compact') that is guaranteed to be numerically comparable.

33.1 Class: ListenerEventIndexer

The listener event indexer class. eventbusclient-message-deprecated-taf-msgs-listener-event-indexer-listenereventindexer-is-special-code -----

Check if a number is code for special purpose. eventbusclient-message-deprecated-taf-msgs-listener-event-indexer-listenereventindexer-is-recovery-code -----

Check if a number is code for recovery mode. eventbusclient-message-deprecated-taf-msgs-listener-event-indexer-listenereventindexer-recovery-code -----

Return special code for recovery mode. eventbusclient-message-deprecated-taf-msgs-listener-event-indexer-listenereventindexer-get-recovery-scope -----

Extract recovery scope from code. eventbusclient-message-deprecated-taf-msgs-listener-event-indexer-listenereventindexer-encode -----

Return the combined index. eventbusclient-message-deprecated-taf-msgs-listener-event-indexer-listenereventindexer-decode -----

Decompose the combined index. eventbusclient-message-deprecated-taf-msgs-listener-event-indexer-listenereventindexer-suite-setup-intro -----

Check if suite setup started. eventbusclient-message-deprecated-taf-msgs-listener-event-indexer-listenereventindexer-suite-setup-outro -----

Check if suite setup completed. eventbusclient-message-deprecated-taf-msgs-listener-event-indexer-listenereventindexer-suite-teardown-intro -----

Check if suite teardown started. eventbusclient-message-deprecated-taf-msgs-listener-event-indexer-listenereventindexer-suite-teardown-outro -----

Check if suite teardown completed. eventbusclient-message-deprecated-taf-msgs-listener-event-indexer-listenereventindexer-test-setup-intro -----

Check if test setup started. eventbusclient-message-deprecated-taf-msgs-listener-event-indexer-listenereventindexer-test-setup-outro -----

Check if test setup completed. eventbusclient-message-deprecated-taf-msgs-listener-event-indexer-listenereventindexer-test-teardown-intro -----

Check if test teardown started. eventbusclient-message-deprecated-taf-msgs-listener-event-indexer-listenereventindexer-test-teardown-outro -----

Check if test teardown completed. eventbusclient-message-deprecated-taf-msgs-listener-event-indexer-listenereventindexer-cycle-index -----

Getter for cycle index. eventbusclient-message-deprecated-taf-msgs-listener-event-indexer-listenereventindexer-suite-

```

state -----
Getter for suite state. eventbusclient-message-deprecated-taf-msgs-listener-event-indexer-listenereventindexer-suite-
index -----
Getter for suite index. eventbusclient-message-deprecated-taf-msgs-listener-event-indexer-listenereventindexer-test-
state -----
Getter for test state. eventbusclient-message-deprecated-taf-msgs-listener-event-indexer-listenereventindexer-test-
index -----
Getter for test index. eventbusclient-message-deprecated-taf-msgs-listener-event-indexer-listenereventindexer-reset-
cycle-state -----
Reset cycle state. eventbusclient-message-deprecated-taf-msgs-listener-event-indexer-listenereventindexer-reset-suite-
state -----
Reset suite state. eventbusclient-message-deprecated-taf-msgs-listener-event-indexer-listenereventindexer-reset-test-
state -----
Reset test state. eventbusclient-message-deprecated-taf-msgs-listener-event-indexer-listenereventindexer-inc-cycle-
state -----
Move up the cycle index to next state. eventbusclient-message-deprecated-taf-msgs-listener-event-indexer-
listenereventindexer-inc-suite-state -----
Move up the suite index to next state. eventbusclient-message-deprecated-taf-msgs-listener-event-indexer-
listenereventindexer-inc-test-state -----
Move up the test index to next state.

```

Chapter 34

listener_event_msg.py

The listener event message module. eventbusclient-message-deprecated-taf-msgs-listener-event-msg-listenereventmsg

=====

The listener event message class.

Chapter 35

`dict_message.py`

DictMessage: A message that can be initialized from a dictionary. eventbusclient-message-dict-message-dictmessage-from-data -----

35.1 Method: `get_value`

Chapter 36

plugin_loader.py

36.1 Class: ConfigValidator

Validates configuration data against a given schema. Raises `ValueError` if validation fails.

36.1.1 Method: `validate`

Validate the configuration against the schema.

Arguments:

- `config`
/ *Condition*: required / *Type*: dict /
Configuration data to validate.

36.2 Class: PluginLoader

The *PluginLoader* class dynamically loads serializers, exchange handlers, and messages.

This class scans specified directories for Python modules, imports them, and registers any classes that match the expected base types (e.g., *Serializer*, *ExchangeHandler*, and *BaseMessage*). It is designed to facilitate the dynamic discovery and use of plugins in the application.

36.2.1 Method: `get_serializer`

Get a serializer class by its name.

Arguments:

- `name`
/ *Condition*: required / *Type*: str /
Name of the serializer class to retrieve.

Returns:

/ *Type*: `Serializer` | `None` /
Serializer class or `None` if not found.

36.2.2 Method: `get_exchange_handler`

Get an exchange handler class by its name.

Arguments:

- `name`
/ *Condition*: required / *Type*: str /
Name of the exchange handler class to retrieve.

Returns:

/ *Type*: `ExchangeHandler` | `None` /
Exchange handler class or `None` if not found.

36.2.3 Method: `get_message`

Get a message class by its name.

Arguments:

- `name`
/ *Condition*: required / *Type*: str /
Name of the message class to retrieve

Returns:

/ *Type*: `BaseMessage` | `None` /
Message class or `None` if not found.

36.2.4 Method: `load_config`

Load configuration from a JSONP file and validate it against the schema.

Arguments:

- `config_path`
/ *Condition*: required / *Type*: str /
Path to the configuration file. If it is a relative path, it will be resolved to an absolute path.

Returns:

/ *Type*: `DotDict` | `None` /
A `DotDict` containing the configuration data if the file exists and is loaded successfully, otherwise `None`.

Chapter 37

publisher.py

AsyncPublisher: Publishes messages to an exchange using aio_pika.

Chapter 38

qlogger.py

38.1 Class: ColorFormatter

Custom formatter class for setting log color.

38.1.1 Method: format

Set the color format for the log.

Arguments:

- `record`
/ *Condition*: required / *Type*: str /
Log record.

Returns:

/ *Type*: logging.Formatter /
Log with color formatter.

38.2 Class: QFileHandler

Handler class for user defined file in config.

38.2.1 Method: get_log_path

Get the log file path for this handler.

Arguments:

- `config`
/ *Condition*: required / *Type*: DictToClass /
Connection configurations.

Returns:

/ *Type*: str /
Log file path.

38.2.2 Method: get_config_supported

Check if the connection config is supported by this handler.

Arguments:

- `config`
/ *Condition*: required / *Type*: DictToClass /
Connection configurations.

Returns:

/ *Type*: bool /
True if the config is supported.
False if the config is not supported.

38.3 Class: QDefaultFileHandler

Handler class for default log file path.

38.3.1 Method: get_log_path

Get the log file path for this handler.

Arguments:

- `logger_name`
/ *Condition*: required / *Type*: str /
Name of the logger.

Returns:

/ *Type*: str /
Log file path.

38.3.2 Method: get_config_supported

Check if the connection config is supported by this handler.

Arguments:

- `config`
/ *Condition*: required / *Type*: DictToClass /
Connection configurations.

Returns:

/ *Type*: bool /
True if the config is supported.
False if the config is not supported.

38.4 Class: QConsoleHandler

Handler class for console log.

38.4.1 Method: get_config_supported

Check if the connection config is supported by this handler.

Arguments:

- `config`
/ *Condition*: required / *Type*: DictToClass /
Connection configurations.

Returns:

/ *Type*: bool /
True if the config is supported.
False if the config is not supported.

38.5 Class: QLogger

Logger class for QConnect Libraries.

38.5.1 Method: get_logger

Get the logger object.

Arguments:

- `logger_name`
/ *Condition*: required / *Type*: str /
Name of the logger.

Returns:

- `logger`
/ *Type*: Logger /
Logger object. .

38.5.2 Method: set_handler

Set handler for logger.

Arguments:

- `config`
/ *Condition*: required / *Type*: DictToClass /
Connection configurations.

Returns:

- `handler_ins`
/ *Type*: logging.handler /
None if no handler is set.
Handler object.

38.5.3 Method: get_handler

Get the handler of the logger.

Chapter 39

rendezvous.py

Chapter 40

base_serializer.py

40.1 Method: serialize

Serialize a message object to bytes.

Arguments:

- `msg`
/ *Condition*: required / *Type*: Any /
Message object to be serialized.

40.2 Method: deserialize

Deserialize bytes back into a message object.

Arguments:

- `data`
/ *Condition*: required / *Type*: bytes /
Serialized message data to be deserialized.

Chapter 41

json_serializer.py

JsonSerializer: Serializes BaseMessage subclasses to JSON strings. Requires message classes to implement:

System Message

ERROR/3 in <string>, line 6
Unexpected indentation. backrefs:

- to_dict()
- from_dict(data: dict)

41.1 Method: serialize

Serialize a message object to JSON bytes.

Arguments:

- msg
/ *Condition*: required / *Type*: BaseMessage /
Message object to be serialized.

Returns:

/ *Type*: bytes /
Serialized message as JSON bytes.

41.2 Method: deserialize

Deserialize bytes back into a message object.

Arguments:

- data
/ *Condition*: required / *Type*: bytes /
Serialized message data in bytes format.
- message_cls
/ *Condition*: required / *Type*: Type[BaseMessage] /
Class of the message to deserialize into.

Returns:

/ *Type*: str /
Deserialized message object of type *message_cls*.

Raises:

- `ValueError` If *message_cls* is not provided.
- `TypeError` If *message_cls* does not implement *from_dict(data: dict)*.
- `RuntimeError` If deserialization fails due to invalid data or other issues.

Chapter 42

pickle_serializer.py

PickleSerializer: Built-in serializer using Python pickle.

WARNING: Pickle is not secure against untrusted data. Only use in trusted environments.

42.1 Method: serialize

Serialize a message object to bytes using pickle.

Arguments:

- msg
/ *Condition*: required / *Type*: Any /
Message object to be serialized.

42.2 Method: deserialize

Deserialize bytes back into a message object using pickle.

Arguments:

- data
/ *Condition*: required / *Type*: bytes /
Serialized message data to be deserialized.

Chapter 43

protobuf_serializer.py

ProtobufSerializer: Serializer using Protocol Buffers.

Requires protobuf message classes generated from .proto files.

43.1 Method: serialize

Serialize a protobuf message object to bytes.

Arguments:

- msg
/ *Condition*: required / *Type*: Message /
Protobuf message object to be serialized.

43.2 Method: deserialize

Deserialize bytes back into a protobuf message object.

Arguments:

- data
/ *Condition*: required / *Type*: bytes /
Serialized protobuf message data to be deserialized.
- message_cls
/ *Condition*: required / *Type*: type /
Protobuf message class to instantiate.

Chapter 44

startup_policy.py

44.1 Function: `resolve_message_cls`

Turn a JSON 'class' string into an actual class object.

Arguments:

- `value`
/ *Condition*: required / *Type*: str or type or None /
The class to resolve. Can be a dotted path, a short name, or a type.
- `base_cls`
/ *Condition*: required / *Type*: type /
The base class that the resolved class must inherit from.
- `registry`
/ *Condition*: optional / *Type*: dict[str, type] / *Default*: None /
Optional registry mapping short names to classes.
- `extra_modules`
/ *Condition*: optional / *Type*: list[str] / *Default*: None /
List of module names to search for the class by short name.
- `default`
/ *Condition*: optional / *Type*: type / *Default*: None /
Default class to use if value is None.

Returns:

/ *Type*: type /
The resolved class object inheriting from `base_cls`.

44.2 Function: `build_policy_from_item`

Build a StartupPolicy from a configuration item.

Arguments:

- `item`
/ *Condition*: required / *Type*: dict /
The configuration item with keys:
 - "class": str - dotted path to the policy class

- "args": dict - optional arguments for the policy class
- "wrap": list - optional list of wrappers, each with "class" and "args"

Returns:

/ *Type*: `StartupPolicy` /

The constructed `StartupPolicy` instance.

44.3 Function: `build_policy_from_config`

Build a `StartupPolicy` from a configuration dictionary.

Arguments:

- `cfg`

/ *Condition*: required / *Type*: dict /

The configuration dictionary. Supports either:

- Legacy single policy:
 - * "startup_policy": str - dotted path to the policy class
 - * "startup_policy_args": dict - optional arguments for the policy class
- New multi-policy:
 - * "startup_policies": list - list of policy items (see `build_policy_from_item`)
 - * "startup_policies_mode": str - optional mode ("sequential" or "parallel")
 - * "startup_policies_fail_fast": bool - optional fail-fast flag (default `True`)

Returns:

/ *Type*: `Optional[StartupPolicy]` /

The constructed `StartupPolicy` instance, or `None` if no policy is configured.

44.4 Class: `StartupPolicy`

A startup policy can delay or block the completion of `EventBusClient.start()` until certain conditions are met. This can be used to implement rendezvous patterns, fixed delays, or other custom logic.

44.5 Class: `NoWait`

No waiting, start immediately.

44.6 Class: `FixedDelay`

44.7 Class: `HandshakeBarrier`

Wait until at least `N` consumers announce ready for the given roles/topics. Example:

System Message

```
ERROR/3 in <string>, line 123
Unexpected indentation. backrefs:
```

```
HandshakeBarrier({"telemetry.*": 1, "orders.created": 2}, timeout=5.0)
```

44.8 Class: `PanelControlLegacyByAlias`

Legacy startup policy for panel control mode based on controller alias.

44.9 Class: `GeneralCacheStartupPolicy`

Example startup policy that decides whether to start a 'general cache' at connect-time, and with which routing keys / message class. Can be alias-aware.

44.10 Class: `ConfigureUnroutablePolicy`

Configure unroutable behavior BEFORE exchange setup. mode: "drop" | "alternate-exchange" | "return"
on_unroutable: "log" | "raise" | "cache" | "callback"

44.11 Class: `PolicyChain`

Apply multiple policies either sequentially or in parallel.

- mode="sequential": run in order; if fail_fast=True, stop on first exception.
- mode="parallel": run concurrently; if fail_fast=True, propagate first exception (gather(..., return_exceptions=False)).

Chapter 45

subscriber.py

AsyncSubscriber: Subscribes to messages from an exchange using aio_pika.

45.1 Method: cache

Get the subscription cache, initializing it if necessary.

Returns:

/ Type: SubscriptionCache /
The subscription cache instance.

45.2 Method: routing_key

45.3 Method: callback

Chapter 46

subscription_cache.py

A thread-safe cache for storing and retrieving items in a FIFO manner. `eventbusclient-subscription-cache-subscriptioncache-register-callback` -----

Register a callback function to be called whenever a new item is added to the cache.

Arguments:

- `callback`
/ *Condition*: required / *Type*: Callable[[T], None] /
A function that takes an item of type T and returns None. This function will be called in a separate thread whenever a new item is added to the cache.

46.1 Method: `append`

Add an item to the cache, possibly dropping the oldest item if full.

Arguments:

- `item`
/ *Condition*: required / *Type*: T /
The item to add to the cache.

46.2 Method: `get`

Block until any item arrives; return it (FIFO) and remove it.

Arguments:

- `timeout`
/ *Condition*: optional / *Type*: Optional[float] / *Default*: None /
Maximum time to wait for an item in seconds. If None, wait indefinitely.

46.3 Method: `pop`

Alias for `get()`.

Arguments:

- `timeout`
/ *Condition*: optional / *Type*: Optional[float] / *Default*: None /
Maximum time to wait for an item in seconds. If None, wait indefinitely.

Returns:

/ Type: T /

The next item from the cache, removed from the cache.

46.4 Method: pop_nothrow

Remove and return the first item in the cache, only if not empty.

Returns:

/ Type: Optional[T] /

The first item from the cache, or None if the cache is empty.

46.5 Method: peek_nothrow

Peek at the first item in the cache, only if not empty.

Returns:

/ Type: Optional[T] /

The first item from the cache, or None if the cache is empty.

46.6 Method: peek

Block until any item arrives; return it (FIFO) but do not remove it.

Arguments:

- `timeout`

/ Condition: optional / Type: Optional[float] / Default: None /

Maximum time to wait for an item in seconds. If None, wait indefinitely.

Returns:

/ Type: T /

The next item from the cache, without removing it.

46.7 Method: wait_for

Block until an item matching the predicate arrives; return it and remove it.

Arguments:

- `predicate`

/ Condition: required / Type: Callable[[T], bool] /

A function that takes an item of type T and returns True if it matches the desired condition.

- `timeout`

/ Condition: optional / Type: Optional[float] / Default: None /

Maximum time to wait for a matching item in seconds. If None, wait indefinitely.

Returns:

/ Type: T /

The first item that matches the predicate, removed from the cache.

46.8 Method: drain

Remove and return up to `max_items` from the cache in FIFO order.

Arguments:

- `max_items`
/ Condition: optional / Type: Optional[int] / Default: None /
 Maximum number of items to remove. If None, remove all items.

Returns:

/ Type: List[T] /
 A list of removed items, in the order they were in the cache.

46.9 Method: peek_last

Peek at the last item in the cache, only if not empty.

Returns:

/ Type: Optional[T] /
 The last item in the cache, or None if the cache is empty.

46.10 Method: wait_for_one

Wait for a single target message to appear in the cache.

Arguments:

- `target`
/ Condition: required / Type: Any /
 The target message to wait for.
- `timeout`
/ Condition: optional / Type: float / Default: 30.0 /
 Maximum time to wait in seconds.

Returns:

/ Type: bool /
 True if the target message was found and removed from the cache, False if the timeout was reached.

46.11 Method: wait_for_many

Wait for multiple target messages to appear in the cache.

Arguments:

- `targets`
/ Condition: required / Type: List[Any] /
 List of target messages to wait for.
- `mode`
/ Condition: required / Type: WaitMode /
 Mode of waiting:

- WaitMode.ALL_IN_GIVEN_ORDER: Wait for all target messages in the given order.
- WaitMode.ALL_IN_RANDOM_ORDER: Wait for all target messages in any order.
- WaitMode.ANY_OF_GIVEN_MSGS: Wait for any one of the target messages.

- timeout

/ Condition: optional / Type: float / Default: 30.0 /
Maximum time to wait in seconds.

Returns:

/ Type: List[int] /

List of indices of the target messages that were found, in the order they were found.

Chapter 47

utils.py

47.1 Class: Singleton

Class to implement Singleton Design Pattern. This class is used to derive the TTFisClientReal as only a single instance of this class is allowed.

Disabled pyLint Messages: R0903: Too few public methods (%s/%s)

System Message

ERROR/3 in <string>, line 9
Unexpected indentation. backrefs:

Used when class has too few public methods, so be sure it's really worth it.

This base class implements the Singleton Design Pattern required for the TTFisClientReal. Adding further methods does not make sense.

System Message

WARNING/2 in <string>, line 13
Block quote ends without a blank line; unexpected unindent. backrefs:

47.2 Class: DictToClass

Class for converting dictionary to class object.

47.2.1 Method: validate

47.3 Class: Utils

Class to implement utilities for supporting development.

47.3.1 Method: get_all_descendant_classes

Get all descendant classes of a class

Arguments: cls: Input class for finding descendants.

Returns:

/ *Type*: list /

Array of descendant classes.

47.3.2 Method: get_all_sub_classes

Get all children classes of a class

Arguments:

- `cls`
/ *Condition*: required / *Type*: class /
Input class for finding children.

Returns:

/ *Type*: list /
Array of children classes.

47.3.3 Method: is_valid_host

47.3.4 Method: caller_name

Get a name of a caller in the format module.class.method

Arguments:

- `skip`
/ *Condition*: required / *Type*: int /
Specifies how many levels of stack to skip while getting caller name. * `skip=1` means "who calls me" * `skip=2` means "who calls my caller" etc.

Returns:

/ *Type*: str /
An empty string is returned if skipped levels exceed stack height

47.3.5 Method: load_library

Load native library depend on the calling convention.

Arguments:

- `path`
/ *Condition*: required / *Type*: str /
Library path.
- `is_stdcall`
/ *Condition*: optional / *Type*: bool / *Default*: True /
Determine if the library's calling convention is stdcall or cdecl.

Returns:

Loaded library object.

47.3.6 Method: is_ascii_or_unicode

Check if the string is ascii or unicode

Arguments: `str_check`: string for checking codecs: encoding type list

Returns:

/ *Type*: bool /
True : if checked string is ascii or unicode
False : if checked string is not ascii or unicode

47.4 Class: Job

47.4.1 Method: stop

47.4.2 Method: run

47.5 Class: ResultType

Result Types.

47.6 Class: ResponseMessage

Response message class

47.6.1 Method: get_json

Convert response message to json

Returns:

Response message in json format

47.6.2 Method: get_data

Get string data result

Returns:

String result

47.6.3 Method: create_from_string

Chapter 48

wait_mode.py

Enumeration for different wait modes.

- ALL_IN_GIVEN_ORDER: Wait for all specified messages in the given order.
- ALL_IN_RANDOM_ORDER: Wait for all specified messages in any order.
- ANY_OF_GIVEN_MSGS: Wait for any one of the specified messages.

Chapter 49

Glossary

RabbitMQ (open-source message broker)

⇒ [homepage](#)

⇒ [documentation for developers](#)

EventBusClient (high-level messaging framework built on top of **RabbitMQ**)

⇒ [homepage](#)

RobotFramework AIO (**AIO** = **A**ll **I**n **O**ne; test automation framework with extended features, based on the **Robot Framework**)

⇒ [RobotFramework AIO homepage](#)

⇒ [Robot Framework homepage](#)

Chapter 50

Appendix

About this package:

Table 50.1: Package setup

Setup parameter	Value
Name	EventBusClient
Version	0.2.0
Date	20.02.2026
Description	IPC message-bus based on RabbitMQ message broker
Package URL	python-rabbitmq-messagebus
Author	Nguyen Huynh Tri Cuong
Email	Cuong.NguyenHuynhTri@vn.bosch.com
License	Apache-2.0
Python required	≥ 3.11

Chapter 51

History

0.1.0	07/2025
<i>Initial version</i>	
0.1.2	01/2026
<i>Fixed bugs and improved stability.</i>	
0.2.0	02/2026
<i>Usage of Setuptools replaced by usage of PIP/TOML/Setuptools</i>	

EventBusClient.pdf*Created at 20.02.2026 - 16:09:46**by GenPackageDoc v. 0.42.0 / 12.01.2026*
